SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA3711**

COURSE OUTCOME COVERED

CO2: ***Design effective test cases using black-box, white-box testing techniques and to control methods flow.***

Assessment Tool Weightage: 30%

Code of Conduct:

I P.Ranjithkumar (192411119)_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Ranjithkumar

1. A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.

1. Aim and assumptions

The aim is to infer withdrawal rules from observed input-output pairs without inspecting source code. For illustration, assume a valid ATM withdrawal must use a positive amount, be a multiple of the machine denomination (for example, 100 units), not exceed a per-transaction limit of 20,000 units, and not exceed the available balance. These are example assumptions, not facts about every ATM.

2. Functional decomposition

Card/session validation → PIN/authentication → account selection → amount validation → limit check → balance check → cash dispense → ledger update/receipt

- Session and card: valid card, active session, permitted retry count.
- Amount validation: numeric, positive, denomination-compatible.
- Policy validation: per-transaction and daily limits.
- Funds validation: available balance, holds, and minimum-balance rule.
- Transaction completion: dispense cash once, debit once, log result, and handle dispenser failure.

3. Equivalence classes and boundaries

Input/rule	Valid class	Invalid classes	Boundary examples
Withdrawal amount	Positive, denomination-compatible, within limits	Zero/negative; non-numeric; wrong denomination; over limit	0, 1, 100, 200, limit, limit+denomination
Available balance	Amount ≤ available balance	Amount > balance; balance unavailable	balance−denomination, balance,

			balance+denomination
Daily total	New total ≤ daily limit	New total > daily limit	remaining allowance−100, exact remaining allowance, +100
Card/session	Active card and authenticated session	Expired/blocked card; invalid PIN; timed-out session	Last permitted PIN attempt; one extra attempt
ATM dispenser	Enough cash and functioning hardware	Cash unavailable; dispenser error	Exact cash available vs one denomination short

4. Likely black-box techniques and adequacy

Observed examples suggest equivalence partitioning (valid/invalid amount groups), boundary-value analysis (limit and balance edges), and error guessing (insufficient funds or dispenser failure). A small collection of normal successful withdrawals is inadequate because it does not establish limit edges, daily aggregation, repeated requests, or consistency between cash dispensing and account debit.

5. Improved test cases

ID	Test input / setup	Expected result	Purpose
ATM-01	Amount 100; balance 1,000; under limits	Success; cash dispensed; balance decreases by 100	Nominal valid case
ATM-02	Amount 0	Reject; clear amount validation message; no debit	Lower boundary
ATM-03	Amount −100	Reject; no debit	Negative class
ATM-04	Amount 150 where denomination is 100	Reject as invalid denomination	Format/business rule
ATM-05	Amount equals transaction limit	Success if balance and daily allowance permit	Upper boundary
ATM-06	Amount one denomination above limit	Reject; no debit	Upper boundary + 1 step
ATM-07	Amount equals	Apply documented	Balance boundary

	available balance	minimum-balance rule; otherwise success	
ATM-08	Amount exceeds available balance by 100	Insufficient-funds error; no debit	Balance boundary
ATM-09	Daily total reaches exact limit	Accept if policy allows exact limit	Daily limit edge
ATM-10	Daily total would exceed limit by 100	Reject; no debit	Daily aggregation
ATM-11	Dispenser fails after authorization	Reversal/compensation; customer not charged permanently	Atomicity
ATM-12	Duplicate request/retry after timeout	At most one debit and one dispense	Idempotency

6. Conclusion

A sound reconstruction combines equivalence partitioning, boundary-value analysis, negative testing, and transaction-integrity checks. Every rejected withdrawal must leave the account unchanged, and every successful withdrawal must debit exactly once.

2. **A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.**

1. Method and assumptions

Because the actual code and traces are not attached, use a representative function to demonstrate CFG reconstruction. The method is to identify basic blocks, connect possible control transfers, mark decisions and loops, then design tests for node, edge/branch, condition, and independent-path coverage.

2. Example logic and CFG

1 Start
 |
2 Read input
 |
3 Is input valid? ----No----> 8 Error → 9 End
 | Yes
4 Initialize total
 |
5 For each item? ----No----> 7 Return total → 9 End
 | Yes
6 Is item eligible? --No--┐
 | Yes |
 v |
6a Add item to total ----┘ (back to 5)

Decision points: (3) input valid, (5) another item exists, and (6) item eligible. The loop is the repeated item-processing edge from 6/6a back to 5. The graph is conceptual; replace nodes with the supplied code's actual statements.

3. Independent paths

- P1: Start → Read → invalid input → Error → End.
- P2: Start → Read → valid → initialize → no items → return → End (zero iterations).
- P3: Valid → initialize → item exists → item ineligible → loop back → no items → return.
- P4: Valid → initialize → item exists → item eligible → add → loop back → no items → return.

Cyclomatic complexity for a connected CFG is $V(G) = E - N + 2P$, where E is edges, N is nodes, and P is connected components (normally 1). Equivalently, for a simple structured graph, V(G) = number of binary decision nodes + 1. Each independent path should contribute at least one new edge.

4. Coverage adequacy

Criterion	Meaning	What it can miss
Statement coverage	Every executable statement runs at least once	A decision's other outcome may never run
Branch/edge coverage	Every decision outcome/edge is taken	Individual Boolean subconditions may not each affect result
Condition coverage	Each atomic Boolean condition is true and false	Does not necessarily cover every combination of conditions
Basis-path coverage	Execute a basis set of linearly independent paths	Loops with unbounded iterations cannot be exhaustively enumerated

5. Reconstructed test plan

Test	Input/setup	Path/coverage target	Expected
CFG-01	Malformed or missing input	Invalid branch	Error handled; no processing
CFG-02	Valid input, empty collection	Valid branch; loop zero times	Return initial total
CFG-03	One ineligible item	Loop true; eligibility false	Total unchanged
CFG-04	One eligible item	Loop true; eligibility true	Item contributes correctly
CFG-05	Several mixed items	Loop repeats; both eligibility outcomes	Correct aggregate
CFG-06	Large collection / termination case	Repeated loop and resource behavior	Completes within expected limits

6. Conclusion

Full path coverage is generally impossible for arbitrary loops because iteration counts can be unbounded. A practical target is basis-path coverage plus loop testing for zero, one, two, typical, and maximum iterations, with branch and condition coverage where risk warrants it.

3. **A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.**

1. Reverse-engineering clues

Defects concentrated at minimum/maximum field lengths suggest boundary-value analysis; failures for invalid usernames, passwords, card numbers, or transaction amounts suggest equivalence partitioning; defects that occur only after a particular sequence of login, verification, and payment suggest state or path testing. A defect log is evidence of failure locations, not definitive proof of the original technique.

2. Testable components

Browser/UI → input validation → authentication/session → authorization → transaction validation → payment/ledger service → database → response/audit log

- Login: username/password formats, lockout, session expiry, and error handling.
- Authorization: role/ownership checks for protected transactions.
- Transaction: amount, currency, balance, duplicate submission, and state transitions.
- Persistence/audit: atomic writes, rollback, audit events, and data consistency.

3. Coverage gaps and reconstructed techniques

Likely clue	Likely technique	Potential gap
Only ordinary login successes tested	Nominal functional tests	No invalid credentials, lockout, expiry, or injection tests
Failures at max field length	BVA	Missing max−1, max, max+1 and Unicode cases
Failures for groups of invalid data	ECP	Classes may be too broad or overlapping
Failure after retries/redirects	Path/state testing	Missing alternate branches and session states
Duplicate charges after refresh	Sequence/idempotency testing	No retry, timeout, or concurrent-request scenarios

4. Example integrated test suite

ID	Test	Technique	Expected result
WEB-01	Valid credentials and permitted transaction	Nominal + path	Authenticated; one valid transaction
WEB-02	Unknown user / wrong password / disabled account	ECP	Safe, consistent rejection; no session
WEB-03	Password length min−1, min, max, max+1	BVA	Only policy-compliant lengths accepted
WEB-04	Repeated failed logins up to and beyond threshold	Boundary + state	Lockout/rate limit follows policy
WEB-05	Expired session submits transaction	Path/state	Re-authentication required; no transaction
WEB-06	Amount 0, minimum valid, maximum valid, max+step	ECP+BVA	Correct accept/reject at each edge
WEB-07	User A accesses User B's transaction	Authorization negative test	Denied; no data leakage
WEB-08	Payment request times out then retries	Sequence/idempotency	One charge maximum; status recoverable
WEB-09	Database failure after authorization	Error path	Rollback or controlled pending state; audit trail
WEB-10	Concurrent requests against same balance	Concurrency/stress	No overspend or duplicate ledger entries

5. Conclusion

A comprehensive strategy combines ECP and BVA for input domains with CFG/path testing for code logic, state-transition testing for sessions and transactions, and security/concurrency tests. Defect patterns guide hypotheses, which must be confirmed by targeted tests.

4. **A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.**

1. Inferred inventory rules

Assume an update request includes item ID, quantity change, and current stock. A typical rule is new stock = current stock + change; stock must not become negative; low-stock alert is triggered when new stock is below a configured reorder threshold. The threshold and whether equality triggers an alert must be confirmed from logs.

2. Functional units

Receive request → validate item and quantity → calculate proposed stock → check stock bounds → commit update → evaluate reorder threshold → emit alert → write audit log

3. Equivalence classes and boundaries

Dimension	Valid classes	Invalid classes / edge cases	Boundary values
Item ID	Existing active SKU	Unknown, inactive, malformed SKU	Known ID vs unknown ID
Quantity change	Integer adjustment permitted by policy	Text, fractional, zero if disallowed	−1, 0, +1
Resulting stock	0 ≤ new stock ≤ capacity	Negative or above capacity	new stock −1, 0, 1; capacity−1, capacity, capacity+1
Low-stock threshold	New stock above threshold; alert condition at/below threshold per policy	Missing/negative threshold	T−1, T, T+1
Concurrent updates	Serialized or safely atomic updates	Lost update / duplicate processing	Two updates reading same initial stock

4. Black-box test cases

ID	Initial stock / request	Expected result	Reason
INV-01	Stock 20; receive +5	Stock becomes 25; no low-stock alert	Normal increase
INV-02	Stock 20; issue −5	Stock becomes 15; alert depends on threshold	Normal decrease
INV-03	Stock 0; issue −1	Reject; stock remains 0	Negative-stock boundary
INV-04	Stock 1; issue −1	Stock becomes 0 if allowed; alert policy applied	Zero boundary
INV-05	Proposed stock T−1	Commit; low-stock alert generated	Below threshold
INV-06	Proposed stock exactly T	Alert or no alert according to documented comparison (< or ≤); verify log evidence	Threshold equality
INV-07	Proposed stock T+1	No low-stock alert	Above threshold
INV-08	Unknown SKU; +10	Reject; no record created	Invalid item class
INV-09	Fractional quantity 1.5	Reject unless fractional stock is explicitly supported	Data type
INV-10	Same request ID replayed	No duplicate stock adjustment	Idempotency
INV-11	Two simultaneous issues of 7 from stock 10	Only one succeeds or both are safely reconciled; stock never negative	Concurrency
INV-12	Database commit succeeds but alert service fails	Stock remains correct; alert retried/queued and observable	Cross-service reliability

5. Conclusion

Logs should be used to infer candidate rules, especially the exact low-stock comparison and transaction atomicity. Boundary cases around zero, capacity, and threshold provide more information than additional ordinary transactions.

5.You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.

1. What the profile can and cannot show

A compiled executable's CPU and memory profile can identify hot functions, repeated allocations, long-running loops, and high-cost branches, but it cannot uniquely reveal source-level decisions. Correlate spikes with symbols, stack traces, profiler call trees, input size, and repeatable workloads. If symbols are unavailable, treat the CFG as conceptual rather than proven.

2. Conceptual CFG for a library module

Start → Parse request → Valid request?

No → Error/return

Yes ↓

Find matching records

↓

For each record? ── No → Build response → End

Yes ↓

Record available?

No → Next record (loop)

Yes ↓

Update loan/reservation state

↓

Write database/log → Next record (loop)

Likely hot regions include searching large catalogues, nested loops over users/books, repeated string comparisons, sorting, serialization, and logging. Memory spikes may indicate large result buffers, unbounded caches, or allocations inside loops.

3. Test strategy for resource-intensive paths

Test	Workload	Measurements and expected result
LIB-01	Empty catalogue; search request	Fast completion; no excessive allocation
LIB-02	One record and one matching record	Correct nominal path and state update
LIB-03	Large catalogue, no match	Search loop terminates; bounded memory
LIB-04	Large catalogue, many matches	Result size capped/paginated; no memory exhaustion
LIB-05	Nested loan/reservation operations	All decision branches exercised; correct availability
LIB-06	Repeated requests over time	No steadily growing memory leak
LIB-07	Peak concurrency and realistic mixed traffic	Latency/error rate remain within service targets
LIB-08	Database/logging service slow or unavailable	Timeout/retry policy works; no corrupt loan state

4. Stability evaluation

- Record baseline CPU, peak/steady-state memory, throughput, p95/p99 latency, error rate, and garbage-collection or allocation data.
- Run load, stress, soak, and spike tests separately; change one major variable at a time.
- Use profiler evidence to prioritize paths, but preserve correctness assertions—fast execution alone is not success.
- Set safe limits for result size, loop iterations, concurrency, and timeouts; verify graceful degradation.

5. Conclusion

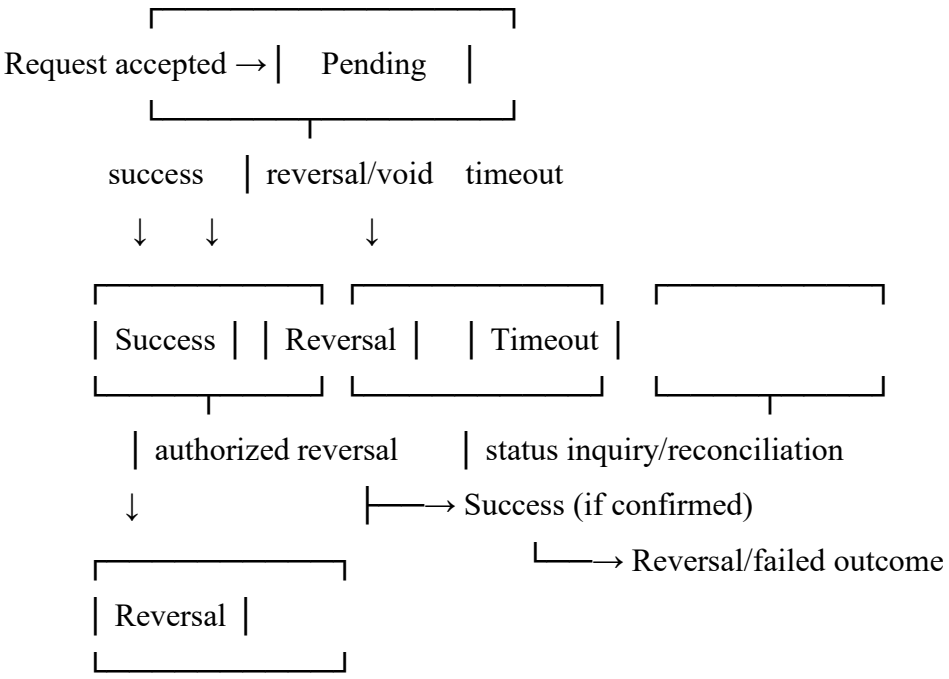
Resource profiles help prioritize white-box path tests but do not prove a CFG. The strongest strategy combines a conceptual graph, profiler-guided instrumentation, loop-boundary tests, stress/soak testing, and functional assertions on every high-cost path.

6.A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.

1. State model and assumptions

Treat payment states as Pending, Success, Reversal, and Timeout. A common safe model permits Pending to resolve to Success, Reversal, or Timeout; Success may be reversed only through an explicit reversal/refund workflow; Timeout is an uncertain outcome and must be reconciled before retrying; terminal states should not silently jump to unrelated states. Actual provider contracts take precedence.

2. State-transition diagram



3. Implicit technique

The observed states and response transitions indicate state-transition testing: test both state coverage (visit each state) and transition coverage (exercise each legal transition). Transition-pair coverage checks consecutive transition pairs. Illegal-transition tests verify that invalid state changes are rejected or safely ignored.

4. Transition test suite

ID	Current state + trigger	Expected next state	Expected checks
PAY-01	New request accepted	Pending	Unique transaction ID; amount/currency validated
PAY-02	Pending + provider approval	Success	One ledger debit; success response persisted
PAY-03	Pending + issuer decline/void	Reversal or documented failed terminal state	No final charge; reversal reference logged
PAY-04	Pending + response timeout	Timeout/unknown	Do not assume failure or success; reconciliation scheduled
PAY-05	Timeout + status query confirms success	Success	No duplicate charge; same transaction ID
PAY-06	Timeout + status query confirms reversal	Reversal	Ledger balanced
PAY-07	Success + authorized reversal request	Reversal	Compensating entry; original record retained
PAY-08	Success + duplicate success callback	Success (idempotent)	No duplicate debit
PAY-09	Reversal + late success callback	Remain Reversal or enter defined reconciliation workflow	No inconsistent final state
PAY-10	Pending + malformed callback	No state change; error logged	Schema/signature validation
PAY-11	Reversal + new success transition without new	Reject/ignore	Illegal transition protection

	transaction		
PAY-12	Unknown transaction ID	No state change; not found response	No data leakage

5. Data constraints and conclusion

- Validate amount > 0, supported currency, transaction ID uniqueness, timestamp, callback signature, and permitted status values.
- Use idempotency keys and atomic ledger updates; callbacks may be duplicated or arrive out of order.

The state model should be verified against the API contract. State and transition coverage, invalid-transition testing, duplicate/out-of-order callbacks, and reconciliation tests are critical for financial correctness.

7.An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.

1. Decomposition

Input parser → schema/type validation → length/range validation → canonicalization → encryption routine → output encoding → error/audit handler

Fuzzing supplies malformed, unexpected, oversized, truncated, and boundary inputs to discover crashes or inconsistent errors. Use an authorized test environment and synthetic data. Fuzzing is a discovery method; BVA and schema-based tests make the results systematic.

2. Boundary and fuzz cases

ID	Input pattern	Expected safe behavior
ENC-01	Each field at minimum permitted length	Accepted if all other constraints pass
ENC-02	Length min−1, min, min+1	Reject below minimum; accept valid edge values
ENC-03	Length max−1, max, max+1	Accept through max; reject oversized input cleanly
ENC-04	Empty, null, missing, wrong type	Clear validation error; no crash
ENC-05	Very large payload within and beyond documented cap	Bounded resource use; oversized payload rejected
ENC-06	Invalid Unicode, embedded NUL, malformed encoding	Consistent rejection or documented normalization
ENC-07	Unexpected delimiters and control characters	Treated as data; no parser confusion or command execution
ENC-08	Repeated random bytes and truncated ciphertext	Safe parse/decrypt failure; no sensitive detail leakage

ENC-09	Correct input repeated many times	Stable memory/CPU; output follows documented determinism/randomization
ENC-10	Output buffer boundary and maximum encoded output	No truncation, overwrite, or partial success

3. Security reasoning

BVA reduces risk by testing just below, at, and just above declared limits. Memory-safe parsing, checked arithmetic, bounded allocations, length validation before copying, parameterized interfaces, and canonicalization reduce overflow and injection risk. Encryption does not itself prevent injection; validation and safe downstream handling are separate responsibilities.

4. Expected evidence

- No crash, hang, unbounded memory growth, or process termination for malformed inputs.
- Errors do not reveal keys, plaintext, stack traces, or internal secrets.
- Invalid input is rejected before encryption or downstream side effects.
- Fuzz failures are minimized to reproducible test cases and added to regression tests.

5. Conclusion

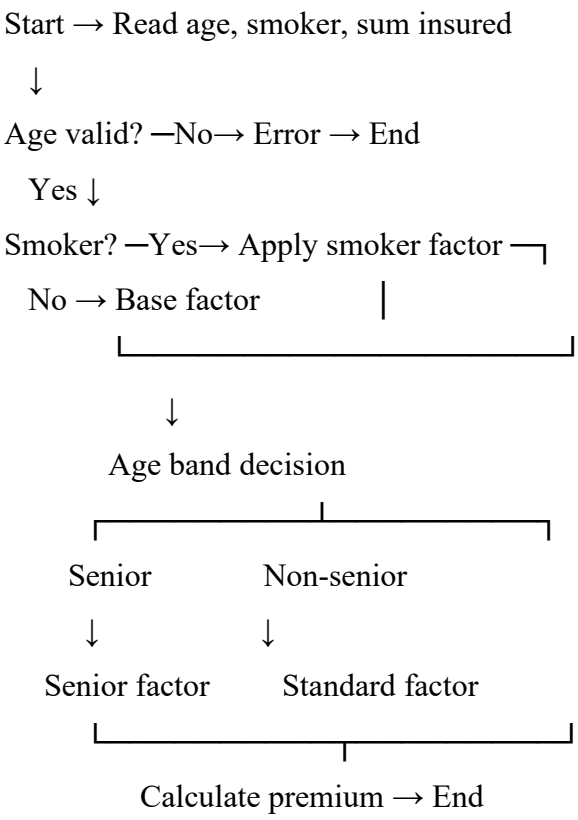
Combine field-by-field equivalence classes, BVA, grammar-aware fuzzing, malformed ciphertext tests, and resource monitoring. Security claims require repeatable evidence and code-level review where available.

8. **A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.**

1. Why 100% statement coverage is insufficient

Statement coverage proves each executable statement ran at least once; it does not prove every branch outcome or Boolean condition was exercised. For example, if (age >= 65 AND smoker == false), a test may execute the premium calculation statement while never independently testing each condition's true/false contribution.

2. Example decision logic and CFG



The diagram illustrates decision clusters. A real CFG must be drawn from the actual source because each else-if, short-circuit operator, and loop changes the node and edge count.

3. Example independent paths and tests

Test	Age	Smoker	Input validity	Expected path
INS-01	Invalid/negative	Any	Invalid	Validation-failure branch
INS-02	Below senior threshold	No	Valid	Non-senior + non-smoker
INS-03	Below senior threshold	Yes	Valid	Non-senior + smoker factor
INS-04	Exactly senior threshold	No	Valid	Senior boundary + non-smoker
INS-05	Exactly senior threshold	Yes	Valid	Senior boundary + smoker factor
INS-06	Above senior threshold	No	Valid	Senior band upper-side
INS-07	Above senior threshold	Yes	Valid	Senior + smoker combination
INS-08	Minimum/maximum sum insured	Both statuses	Valid edge amounts	Premium arithmetic boundaries

4. Coverage criteria and strategy

- Statement coverage: every statement executes at least once.
- Branch coverage: every decision outcome is taken.
- Condition coverage: each atomic condition is true and false at least once.
- Modified Condition/Decision Coverage (MC/DC), where required: show each condition can independently change the overall decision outcome.
- Basis-path testing: calculate cyclomatic complexity and execute a basis set of independent paths; test loop zero/one/multiple iterations if loops exist.

Strict 100% path coverage is not generally feasible when loops can iterate arbitrarily or paths are infeasible. A defensible goal is 100% branch coverage plus condition/MC/DC coverage for high-risk compound decisions, and basis-path coverage.

5. Conclusion

The old tests likely optimized statement coverage. Adding branch, condition, and basis-path tests exposes errors in age bands, smoker factors, boundary comparisons, and premium calculations. Each test should assert both the chosen branch and the calculated numeric result.

9. **A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.**

1. Likely equivalence partitions

Lost-file reports clustered around large files, particular file types, or slow networks suggest the original tests may have treated each dimension too broadly. Each dimension should be partitioned independently and then combined in risk-based pairwise or targeted interaction tests.

Dimension	Candidate partitions	Boundaries / risks
File size	Zero-byte; small; normal; large; above supported maximum	Chunk size−1, exact chunk, chunk+1; maximum−1/max/max+1
File type	Supported common; supported special; unsupported; malformed extension/MIME mismatch	Case sensitivity, Unicode names, very long names
Network	Fast/stable; slow; intermittent; disconnect; high latency	Timeout threshold−1, threshold, threshold+1
Sync state	New upload; changed file; rename; delete; conflict	Simultaneous edits, clock skew, retry and resume
Storage/quota	Below quota; exactly quota; over quota	Available bytes around file size

2. Revised test strategy

ID	Scenario	Expected result
SYNC-01	Small supported file on stable network	Upload/download matches source hash
SYNC-02	Zero-byte file	File exists with correct zero-byte content
SYNC-03	Size at chunk−1, chunk, chunk+1	Correct chunking and reassembly; no missing bytes
SYNC-04	Maximum supported size and one byte over	Correct acceptance at cap; clear rejection above cap
SYNC-05	Slow network with pause/resume	Transfer resumes from verified offset without corruption
SYNC-06	Disconnect during final chunk or metadata commit	Atomic completion or recoverable retry; no false success
SYNC-07	Unsupported type or misleading extension	Policy applied consistently; no silent loss
SYNC-08	Same filename changed on two devices	Conflict detected and resolved per policy; neither version silently disappears
SYNC-09	Rename/delete during offline period	Operations reconcile in correct order
SYNC-10	Large batch of mixed file sizes under concurrent sync	Bounded memory, acceptable latency, no missing/duplicate files
SYNC-11	Checksum mismatch after transfer	Retry or quarantine; never mark corrupt file synchronized
SYNC-12	Quota exactly reached / exceeded	No partial silent write; user receives clear status

3. Stress and data-integrity checks

- Compare cryptographic hashes before and after transfer; verify metadata, timestamps, and file counts.
- Run soak tests and network fault injection: latency, packet loss, brief disconnects, and reconnects.
- Test concurrent edits, retry idempotency, partial uploads, and recovery after process restart.
- Use durable journals/checkpoints and explicit states such as queued, transferring, verified, and committed.

4. Conclusion

ECP identifies meaningful file, network, and state classes; BVA targets size, quota, chunk, and timeout edges; stress and fault-injection tests exercise combinations. Integrity assertions and recovery testing are essential to preventing silent data loss.

10.You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.

1. Reverse-engineering query behavior

Compare generated SQL and returned rows while changing one filter at a time. This reveals whether filters combine with AND, OR, or NOT, how nulls are handled, whether text matching is exact/prefix/substring, and whether pagination/sorting changes results. Use a fixed test dataset with known expected results.

2. Example decision model

Assume filters are: category selected (C), price within range (P), in stock (S), and keyword matches (K). Example intended logic: return a product if (C is not selected OR category matches) AND (P is not selected OR price is in range) AND (S is false OR stock > 0) AND (K is empty OR keyword matches). Actual requirements must be inferred from SQL and product results.

User filters → Validate/normalize parameters → Build predicates
→ Combine predicates (AND/OR/NOT)
→ Execute parameterized SQL → Sort/page → Return rows

3. Decision table

Rule	Category filter	Price filter	In-stock only	Keyword	Expected result condition
R1	Off	Off	No	Empty	All products allowed by default visibility
R2	On	Off	No	Empty	Category matches
R3	Off	On	No	Empty	Price is within inclusive/exclusive range per spec
R4	Off	Off	Yes	Empty	Stock quantity > 0
R5	Off	Off	No	Present	Keyword predicate matches

R6	On	On	Yes	Present	All active filters satisfied if combination is AND
R7	On	On	No	Present	Category AND price AND keyword conditions
R8	Any	Invalid range	Any	Any	Validation error or documented empty result
R9	Any	Any	Any	SQL metacharacters in keyword	Treated as literal data; no SQL injection
R10	No matches	Any	Any	Any	Empty result set, not an error

4. Solved example

Suppose the fixed dataset contains: A = category Books, price 20, stock 4; B = Books, price 60, stock 0; C = Games, price 30, stock 2. Filters are category=Books, price 10–50 inclusive, and in-stock-only=true. Under AND semantics, only A satisfies all three predicates. B fails price and stock; C fails category. Expected result: {A}. If the returned set includes B or C, the generated predicate grouping or filter semantics differ from the assumed rule.

5. Completeness and test cases

ID	Filter combination / input	Expected result
SQL-01	No filters	Default visible catalogue
SQL-02	Each filter alone	Only rows satisfying that filter
SQL-03	All filters together	Intersection under AND semantics
SQL-04	Category A OR category B if multi-select is supported	Rows in either category
SQL-05	Explicit exclusion/NOT filter	Rows not matching excluded condition
SQL-06	Minimum price = maximum price	One price point included/excluded per specification
SQL-07	Null, empty, missing, malformed parameters	Consistent validation and null semantics
SQL-08	Keyword containing quotes, wildcard, or SQL syntax	Parameterized query; no injection; literal/documented matching
SQL-09	Sort ascending/descending with ties	Correct order and deterministic tie-break
SQL-10	Page boundary and empty final page	No duplicate/missing rows across pages

6. Conclusion

Decision tables expose combinations that ad hoc tests miss. Full combinatorial coverage grows exponentially, so use all single-filter tests, all high-risk combinations, pairwise coverage for broad combinations, and explicit tests for AND/OR grouping, NOT, nulls, ranges, sorting, and pagination. Confirm the model against a controlled dataset and generated SQL.

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation